

SIMATS ENGINEERING ASSESSMENT TOOL 2

**COURSE CODE: CSA14
DESIGN**

COURSE NAME: COMPILER

COURSE OUTCOME COVERED

CO3: Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)

Assessment Tool Weightage: 20%

QUESTIONS:

Total Marks:100

Annexure A

Debugging

Code of Conduct:

I SHEIK MOHAMED M_192511145 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
SHEIK MOHAMED M/

Name: Sheikh Mohamed M

Reg NO: 192511145

Course: compiler design

Course code: CSA1408

CO3- Assessment Tool-2

Debugging Tool:

1)

Error:

The Semantic rule associated with the production $E \rightarrow E_1 + T$ is incorrectly defined as

$$\{ E.val = T.val \}$$

→ Synthesized attributes are computed from the attribute values of a node's children.

→ In the incorrect rule $\{ E.val = T.val \}$, the semantic action completely ignores the values synthesized by E_1 ($E_1.val$).

→ During evaluation of $3+5$, E_1 evaluates to 3 and T evaluates to 5. The rule discards 3 and assigns $E.val = 5$.

Corrected Semantic Rule:

$$E \rightarrow E_1 + T \quad \{ E.val = E_1.val + T.val \}$$

Correct Value Computation

- 1) $T \rightarrow \text{num} \Rightarrow T.\text{val} = 3$
- 2) $E_1 \rightarrow T \Rightarrow E_1.\text{val} = T.\text{val} = 3$
- 3) $T \rightarrow \text{num} \Rightarrow T.\text{val} = 5$
- 4) $E \rightarrow E_1 + T \Rightarrow E.\text{val} = 3 + 5 = 8$

2 Error:

The syntax tree constructed places the addition operator (+) below the multiplication operator (*).

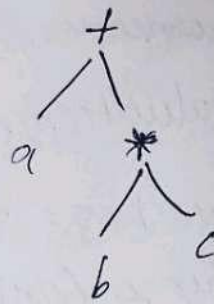
* In an Abstract Syntax Tree (AST), subtrees located at lower depth levels are evaluated first.

* Consulting, constructing + lower than * assigns higher precedence to addition.

* consequently the compiler evaluates $a+b$ first generating $t_1 = (a+b)$ and then multiplies by $(t_2 = t_1 * c)$.

* Standard programming language grammar dictates that multiplication * has higher precedence than addition +.

Corrected syntax tree:



Correct Intermediate Representation:

$$t_1 = b * c;$$

$$t_2 = a + t_1;$$

3) Errors:

The compiler incorrectly computes the root attribute as $E.val = 35$ for the expression $3 + 4 * 5$.

Parse tree & Attribute tree:

$$1) F \rightarrow num(3) \Rightarrow F.val = 3$$

$$T_1 \rightarrow F \Rightarrow T_1.val = 3$$

$$E_1 \rightarrow T_1 \Rightarrow E_1.val = 3$$

$$2) F \rightarrow num(4) \Rightarrow F.val = 4;$$

$$T_2 \rightarrow F \Rightarrow T_2.val = 4$$

$$3) F \rightarrow num(5) \Rightarrow F.val = 5;$$

$$4) T_3 \rightarrow T_2 * F \Rightarrow T_3.val = 4 \times 5 = 20$$

$$5) E_2 \rightarrow E_1 + T_3 \Rightarrow E_2.val = 3 + 20 = 23$$

Root cause:

The erroneous result occurred because the semantic evaluation performed a left-to-right evaluation $(3+4) \times 5 = 35$, disregarding the operator hierarchy imposed by the context-free grammar $(E \rightarrow E + T \mid T, T \rightarrow T * F \mid F)$

Correct Result:

$$E.val = 23$$

Q4

Error:

The compiler raised a false positive Type error: incompatible operands for the expression $x = a + b * c$.

* operand types:

a:	int
b:	float
c:	int
d:	float

Subexpression $(b * c)$:

operands are float and int. Type hierarchy requires widening c from int to float via implicit coercion (int to float (c)).

Resulting type: float.

* Subexpression $2(a + (b * c))$:

Operands are a (int) and the result of $(b * c)$ (float). a must be implicitly converted to float (int to $\text{float}(a)$). Resulting type: float .

* Assignment ($x = \text{expression}$): Target Variable x is float and expression evaluates to float . Types match without narrowing loss.

* The Statement is type valid but Error occurred because the type checker lacked implicit type promotion conversion logic.

Q5 Error:

The type checker incorrectly flagged both assignment as valid without inserting mandatory implicit conversion instruction nodes into the AST.

* Statement 1 ($x = a + b$ where $a: \text{int}$, $b: \text{float}$, $x: \text{int}$);

1) Addition ($a + b$);

operand a (int) must be promoted to float (int to float) (a) before addition. Addition yields float .

2) Assignment ($x = \text{result}$)

Assigning a float result to an int target (x) requires explicit or implicit narrowing conversion (float to int) to prevent unsafe type assignment.

* Statement 2 ($b = a + x$ where $a : \text{int}$, $x : \text{int}$, $b : \text{float}$):

1) Addition ($a + x$):

Both operands are int, yielding an int result.

2) Assignment ($b = \text{result}$):

Assigning an int result to a float (b) requires implicit widening conversion (int to float).

$\therefore$ The type checker must transform expression into an intermediate representation containing explicit conversion nodes.